

Qiskit v2.x ポケットリファレンス

固定知識 + version付きAPI早見表

基準: Qiskit 2.5.2 / qiskit-ibm-runtime 0.49.0

確認日: 2026-09-12 JST

この資料は、試験で即答したい知識を「見た形から復元できる」ように圧縮した参照表です。数式はLaTeX形式で統一し、厳密等式とglobal phaseまでの同値を明確に分けています。

0. 最初に固定する記法

厳密等式とglobal phaseまでの同値

- $|\psi\rangle = |\phi\rangle$: 成分が完全に同じ状態ベクトル
- $U = V$: 成分が完全に同じ行列
- $|\psi\rangle \sim |\phi\rangle$: 同じphysical pure state
- $U \sim V$: 単独作用ではglobal phaseだけ異なる

$$|\psi\rangle = e^{i\alpha}|\phi\rangle \quad U = e^{i\alpha}V$$

覚え方: 「厳密」は係数まで一致。「 $\sim$ 」は状態全体に共通の位相だけを無視する。

$$R_z(\pi) = -iZ \quad R_z(\pi) \sim Z \quad Z|+\rangle = |-\rangle$$

注意: controlled- U では、 U 全体の位相がcontrol分岐間のrelative phaseになることがある。「global phaseだから常に捨ててよい」とは限らない。

複素数の最小暗記

$$i^0 = 1, \quad i^1 = i, \quad i^2 = -1, \quad i^3 = -i, \quad i^4 = 1$$

$$e^{i\theta} = \cos \theta + i \sin \theta$$

θ	$e^{i\theta}$
0	1
$\pi/2$	i
π	-1
$3\pi/2$	$-i$

演算順序

$$U_2U_1|\psi\rangle = U_2(U_1|\psi\rangle)$$

視線の向き: ketに近い右端から作用する。Qiskitで `qc.x(0)`; `qc.z(0)` と追加したoperatorは ZX 。

1. 1量子ビット状態

状態	状態ベクトル	固有状態
$ 0\rangle$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	Z の $+1$
$ 1\rangle$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	Z の -1
$ +\rangle$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$	X の $+1$
$ -\rangle$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$	X の -1
$ +i\rangle$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ i \end{bmatrix}$	Y の $+1$
$ -i\rangle$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -i \end{bmatrix}$	Y の -1

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1$$

$$|\psi(\theta, \phi)\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle$$

Bloch vectorは「各Pauli方向へ落とした影」:

$$\langle X \rangle = \sin \theta \cos \phi, \quad \langle Y \rangle = \sin \theta \sin \phi, \quad \langle Z \rangle = \cos \theta$$

2. 基本ゲート行列

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

位相ゲート S / T

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \quad S^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & -i \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \quad T^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-i\pi/4} \end{bmatrix}$$

$$X^2 = Y^2 = Z^2 = H^2 = I, \quad S^2 = Z, \quad T^2 = S$$

形で覚える: X は成分を交換、 Z は下成分の符号反転、 Y は交換に $\pm i$ が付く。

3. Pauli作用表

computational basis

入力	X	Y	Z	H
$ 0\rangle$	$ 1\rangle$	$i 1\rangle$	$ 0\rangle$	$ +\rangle$
$ 1\rangle$	$ 0\rangle$	$-i 0\rangle$	$- 1\rangle$	$ -\rangle$

X/Y basis

入力	X	Y	Z
$ +\rangle$	$ +\rangle$	$-i -\rangle$	$ -\rangle$
$ -\rangle$	$- -\rangle$	$i +\rangle$	$ +\rangle$
$ +i\rangle$	$i -\rangle$	$ +i\rangle$	$ -\rangle$
$ -\rangle$	$-i +i\rangle$	$- -\rangle$	$ +i\rangle$

迷ったときの判断ルート: ketをcolumn vectorに戻す → 行列を左から掛ける → 最後にだけ global phaseを判定する。

4. Pauli積と反交換

$$X^2 = Y^2 = Z^2 = I$$

巡回順は $+i$

$X \rightarrow Y \rightarrow Z \rightarrow X$ の向き:

$$XY = iZ, \quad YZ = iX, \quad ZX = iY$$

逆向きは $-i$:

$$YX = -iZ, \quad ZY = -iX, \quad XZ = -iY$$

異なるPauliは順序交換で符号反転:

$$XZ = -ZX, \quad XY = -YX, \quad YZ = -ZY$$

$$[X, Y] = 2iZ, \quad [Y, Z] = 2iX, \quad [Z, X] = 2iY$$

指で辿る: $X \rightarrow Y \rightarrow Z$ の時計回りなら $+i$ 、逆なら $-i$ 。

5. 基底変換と共役変換

HadamardはX軸とZ軸を交換

$$HXH = Z, \quad HZH = X, \quad HYH = -Y$$

Phase gateはX軸をY軸へ回す

$$SX S^\dagger = Y, \quad SY S^\dagger = -X, \quad SZ S^\dagger = Z$$

測りたいbasis	測定直前の操作	最後の測定
Z	なし	computational basis
X	H	computational basis
Y	$S^\dagger$ 、その後 H	computational basis

$$|\psi\rangle \xrightarrow{S^\dagger} \xrightarrow{H} \text{Z measurement}$$

逆変換で測る: 測りたい固有basisをcomputational basisへ戻してから測定する。

6. 回転ゲートと位相

Pauli $P \in \{X, Y, Z\}$ に対して:

$$R_P(\theta) = e^{-i\theta P/2} = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} P$$

$\theta = \pi$ ではcos項が消え、 $-iP$ だけが残る:

$$R_x(\pi) = -iX, \quad R_y(\pi) = -iY, \quad R_z(\pi) = -iZ$$

$$R_z\left(\frac{\pi}{2}\right) = e^{-i\pi/4}S, \quad R_z\left(\frac{\pi}{4}\right) = e^{-i\pi/8}T$$

$$P(\phi) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{bmatrix} = e^{i\phi/2} R_z(\phi)$$

したがって $Z \sim R_z(\pi)$ 、 $S \sim R_z(\pi/2)$ 、 $T \sim R_z(\pi/4)$ だが、厳密等式ではない。

7. 期待値を即答する

$$\langle O \rangle = \langle \psi | O | \psi \rangle$$

固有状態なら行列積を省略できる:

$$O|\psi\rangle = \lambda|\psi\rangle \implies \langle O \rangle = \lambda$$

Pauli observableの固有値は ± 1 なので、期待値は $[-1, 1]$ 。

状態	$\langle X \rangle$	$\langle Y \rangle$	$\langle Z \rangle$
$ 0\rangle$	0	0	+1
$ 1\rangle$	0	0	-1
$ +\rangle$	+1	0	0
$ -\rangle$	-1	0	0
$ +i\rangle$	0	+1	0
$ -i\rangle$	0	-1	0

積状態 × 積observableなら期待値も積に分解:

$$\langle a, b | A \otimes B | a, b \rangle = \langle a | A | a \rangle \langle b | B | b \rangle$$

$$\langle ++ | X \otimes X | ++ \rangle = (+1)(+1) = +1$$

$$O = \sum_j c_j P_j \implies \langle O \rangle = \sum_j c_j \langle P_j \rangle$$

8. 測定・counts・shots

$$\Pr(x) = |\langle x | \psi \rangle|^2$$

global phaseは絶対値二乗で消えるが、relative phaseは基底変換後の干渉に影響する。

$$\widehat{\Pr}(x) = \frac{N_x}{N}$$

$$\widehat{\langle Z \rangle} = \frac{N_0 - N_1}{N}$$

$$\widehat{\langle ZZ \rangle} = \frac{N_{00} + N_{11} - N_{01} - N_{10}}{N}$$

bitを $0 \mapsto +1, 1 \mapsto -1$ に変換し、Pauli stringではその積を平均する。 ZZ なら偶数parityが $+1$ 、奇数parityが -1 。

$$\sigma_{\hat{p}} \approx \sqrt{\frac{p(1-p)}{N}}, \quad \sigma_{\hat{P}} \approx \sqrt{\frac{1 - \langle P \rangle^2}{N}}$$

shotsを増やすと標本誤差は減るが、hardware noiseやsystematic biasは自動的に消えない。

9. Qiskitのbit・qubit ordering

1つのQuantumRegisterの場合

$$|q_{n-1} \cdots q_1 q_0\rangle$$

- ket label: 右端が q_0
- statevector: index x は x のbinary表現に対応
- Pauli label: 右端が q_0
- classical string: 右端が通常 c_0

index	ket	(q_1, q_0)
0	$ 00\rangle$	$(0, 0)$
1	$ 01\rangle$	$(0, 1)$
2	$ 10\rangle$	$(1, 0)$
3	$ 11\rangle$	$(1, 1)$

Pauli label "xz":

$$\text{Pauli}(XZ) = X_{q_1} \otimes Z_{q_0}$$

右端アンカー: ketもPauli labelも、まず右端を q_0 に固定する。countsはclassical bit表示なので、qc.measure(qubit, clbit) のmappingを別途追跡する。

10. 多量子ビットゲート

CX

概念上の $|c, t\rangle$ 表記:

$$|c, t\rangle \longmapsto |c, t \oplus c\rangle$$

入力	出力
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 11\rangle$
$ 11\rangle$	$ 10\rangle$

CZ

$$CZ|a, b\rangle = (-1)^{ab}|a, b\rangle$$

$|11\rangle$ だけにminus signを付け、computational-basis bitは反転しない。

SWAP / Toffoli

$$\text{SWAP}|a, b\rangle = |b, a\rangle$$

$$CCX|a, b, t\rangle = |a, b, t \oplus ab\rangle$$

$$CX^2 = CZ^2 = \text{SWAP}^2 = I$$

$$\text{SWAP}_{a,b} = CX_{a,b}CX_{b,a}CX_{a,b}$$

$$CZ = (I \otimes H)CX(I \otimes H)$$

絵として覚える: CXはtarget bitを反転、CZは $|11\rangle$ の符号だけを反転。

11. Bell状態

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \quad |\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$$

$$|\Psi^+\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}}, \quad |\Psi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}$$

$$|00\rangle \xrightarrow{H_{q_0}} \xrightarrow{CX_{q_0,q_1}} |\Phi^+\rangle$$

computational-basis測定では 00 と 11 が各 1/2。各qubit単独はランダムでも、2つは相関する。

状態	$\langle XX \rangle$	$\langle YY \rangle$	$\langle ZZ \rangle$
Φ^+	+1	-1	+1
Φ^-	-1	+1	+1
Ψ^+	+1	+1	-1
Ψ^-	-1	-1	-1

12. 回路構築APIカード

```
from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from qiskit.circuit.library import XGate

qc = QuantumCircuit(2, 2)
theta = Parameter("theta")
qc.h(0)
qc.ry(theta, 0)
qc.cx(0, 1)
qc.measure(0, 1)                # measure(qubit, clbit)

bound = qc.assign_parameters({theta: 0.5})

combined = qc.compose(other)     # default inplace=False
qc.compose(other, inplace=True)
controlled_x = XGate().control()
inverse_gate = some_gate.inverse()
```

`append` はinstruction/gateと対応するqargs/cargsを指定する。`compose` は回路同士の合成に向く。

Dynamic circuitの最小形:

```
qc.measure(0, 0)
with qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
```

SDKで表現できることと、選択したhardware/serviceで実行可能なことは別。current feature supportを確認する。

13. 可視化の使い分け

目的	代表API	主な入力
circuit diagram	<code>qc.draw("mpl")</code>	QuantumCircuit
countsの棒グラフ	<code>plot_histogram</code>	counts
probability distribution	<code>plot_distribution</code>	dict/distribution
qubitごとのBloch球	<code>plot_bloch_multivector</code>	state
basis成分の確率と位相	<code>plot_state_qsphere</code>	state
density matrixの成分	<code>plot_state_city</code>	state/density matrix
状態ベクトル取得	<code>Statevector.from_instruction(qc)</code>	measurementなしのcircuit

分類の軸: API名を丸暗記せず、入力がcircuit・state・countsのどれかで振り分ける。

ここからversion依存: Qiskit APIカード

以下はQiskit 2.5.2 / qiskit-ibm-runtime 0.49.0を基準にした早見表。試験直前と実装前に公式documentationを再確認する。

14. Transpile・ISA・layout

```
abstract circuit
  -> preset pass manager / transpilation
ISA circuit for target backend
  -> observable.apply_layout(isa_circuit.layout)
ISA observable
  -> Runtime primitive
```

```
from qiskit.transpiler import generate_preset_pass_manager

pm = generate_preset_pass_manager(
    backend=backend,
    optimization_level=1,
)
isa_circuit = pm.run(qc)
isa_observable = observable.apply_layout(isa_circuit.layout)
```

段階	役割
basis translation	targetが持つinstructionへ分解
layout	logical qubitをphysical qubitへ対応付け
routing	connectivityを満たすようSWAP等を挿入
optimization	等価性を保ちながら回路改善を試みる

optimization_level は0から3。高いlevelでも特定のdepthやgate countの削減を保証しない。

15. SamplerとEstimator

項目	Sampler V2	Estimator V2
目的	classical outputをsample	observableの期待値を推定
核となる入力	circuit	circuit + observables
追加入力	parameter values, shots	parameter values, precision
local reference	StatevectorSampler	StatevectorEstimator
Runtime	SamplerV2	EstimatorV2
主結果	classical registerごとのBitArray	data.evs、uncertainty情報

```
from qiskit.primitives import StatevectorSampler, StatevectorEstimator
from qiskit_ibm_runtime import SamplerV2, EstimatorV2
```

PUB形:

```
Sampler: (circuit, parameter_values?, shots?)
Estimator: (circuit, observables, parameter_values?, precision?)
```

位置を飛ばす場合は `None` を残す:

```
sampler_pub = (qc, None, 128)
estimator_pub = (qc, observable, None, 0.01)
```

`run()` へは1つ以上のPUBからなるlistを渡す。複数PUBと、1つのPUB内でvectorizeされた入力を区別する。

16. Estimator broadcasting

parameter valuesの最後の軸はcircuit parameterの軸。その軸はbroadcast結果shapeに含めない。

parameter values : (3, 1) $\longrightarrow$ binding : (3)

observables : (2, 1), evs : (2, 3)

意味: 2種類のobservableを3組のparameter valuesで評価する。

判断ルート:

1. parameter valuesの最後のparameter軸を外す。
2. observables shapeとbinding shapeを右揃えする。
3. 各軸が同じ、または一方が1ならbroadcast可能。
4. broadcast後のshapeが `evs` のshape。

17. Primitive result階層

```
primitive.run(pubs)
  -> job
job.result()
  -> PrimitiveResult
PrimitiveResult[i]
  -> PubResult
PubResult.data
  -> Sampler: register名のBitArray
  -> Estimator: evs / stds等
PubResult.metadata
  -> implementation・option依存の補助情報
```

```
result = job.result()
pub_result = result[0]
counts = pub_result.data.meas.get_counts()
```

`meas` はclassical register名に由来する。register名を変えればfield名も変わり得る。

18. Runtime execution modes

mode	向<workload	要点
Job	単発primitive request	primitive^ mode=backend
Session	結果依存の反復multi-job	active window中の専有・予測可能性
Batch	独立した複数job	classical preprocessingを効率化可能

Session/Batchでも最初のjobのqueue待ちは消えない。Batch内jobはsubmit順の実行を保証しない。利用planによる制限も確認する。

```
from qiskit_ibm_runtime import Session, Batch, SamplerV2

sampler_job_mode = SamplerV2(mode=backend)

with Session(backend=backend) as session:
    sampler_session = SamplerV2(mode=session)

with Batch(backend=backend) as batch:
    sampler_batch = SamplerV2(mode=batch)
```

19. Runtime options・noise対策

```
sampler.options.default_shots = 500
sampler.options.dynamical_decoupling.enable = True
estimator.options.resilience_level = 1
```

用語	主目的
dynamical decoupling	idle時間のnoise suppression
measurement mitigation	readout errorの影響を低減
twirling	coherent errorをstochastic化する方向へ変換
ZNE	複数noise factorからzero-noise値を外挿

resilience_level はpreset。個別optionで上書きできるため、level番号だけから最終設定を断定しない。option path、preset内容、dynamic circuitとのcompatibilityはversion-sensitive。

20. Job・Result・統計API

```
job = primitive.run(pubs)
status = job.status()
result = job.result()

old_job = service.job(job_id)
jobs = service.jobs(limit=10)
```

対象	見るもの
lifecycle	job.status()
completed result	job.result()
Sampler outcomes	BitArray.get_counts()
Estimator values	PubResult.data.evs
uncertainty	data.stdts 等。実装・resilience設定を確認
auxiliary context	PubResult.metadata

result() は完了までblockし得る。metadata fieldはimplementation/versionに依存する。

21. OpenQASM 3最小構文

```
OPENQASM 3.0;
include "stdgates.inc";

qubit[2] q;
bit[2] c;

reset q;
h q[0];
cx q[0], q[1];
c = measure q;
```

単bitへの代入:

```
c[0] = measure q[0];
```

Qiskitとの相互変換:

```
from qiskit import qasm3

text = qasm3.dumps(qc)           # circuit -> string
qasm3.dump(qc, stream)          # circuit -> file-like stream
qc2 = qasm3.loads(text)         # string -> circuit
qc3 = qasm3.load(filename)      # file -> circuit
```

OpenQASM 3 importer!には qiskit-qasm3-import が必要。language specification、Qiskitでのparse/representation/export、IBM hardwareでのexecution supportは別々に確認する。

22. 頻出6問の最終確認

Pauli Yの作用: Y を $|0\rangle$ に行列積として作用させ、係数 i を省略しない。

$$Y|0\rangle = i|1\rangle$$

積observableの期待値: $|+\rangle$ は X の固有値 $+1$ の固有状態なので、2量子ビットでも固有値を掛け合わせる。

$$\langle ++ | X \otimes X | ++ \rangle = +1$$

Pauli積の符号: 異なるPauliは反交換し、巡回順と逆向きでは i の符号が反転する。

$$XZ = -ZX = -iY$$

状態ベクトルの厳密等式: Z は $|+\rangle$ の $|1\rangle$ 成分だけを符号反転し、厳密に $|-\rangle$ へ移す。

$$Z|+\rangle = |-\rangle$$

回転ゲートとglobal phase: $R_z(\pi)$ は Z とglobal phaseまで同値だが、厳密な行列等式では係数 $-i$ が付く。

$$R_z(\pi) = -iZ$$

Hadamardによる基底変換: H で挟む共役変換により、 Z operatorは X operatorへ移る。

$$HZH = X$$

30秒チェック

1. 厳密等式か、global phaseまでの同値か。
2. 行列積の右端から作用したか。
3. i の符号をPauli巡回順で確認したか。
4. ket、Pauli label、countsの右端が何を表すか。
5. SamplerとEstimatorの入力・結果を混同していないか。
6. Runtime/API事項にはversion依存性がないか。

公式参照

- IBM Quantum, Primitive inputs and outputs:
<https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>
- IBM Quantum, Estimator inputs and outputs:
<https://quantum.cloud.ibm.com/docs/en/guides/estimator-input-output>
- IBM Quantum, Execution modes:
<https://quantum.cloud.ibm.com/docs/en/guides/execution-modes>
- IBM Quantum, OpenQASM 3 and Qiskit:
<https://quantum.cloud.ibm.com/docs/en/guides/interoperate-qiskit-qasm3>
- IBM Quantum, OpenQASM 3 feature table:
<https://quantum.cloud.ibm.com/docs/en/guides/qasm-feature-table>
- Qiskit Practice Bank objectives map: [../practice-bank/exam-objectives-map.md](#)
- Qiskit Practice Bank Mock 01: [../practice-bank/mock-exams/mock-01.md](#)